SOP: Building detection rules

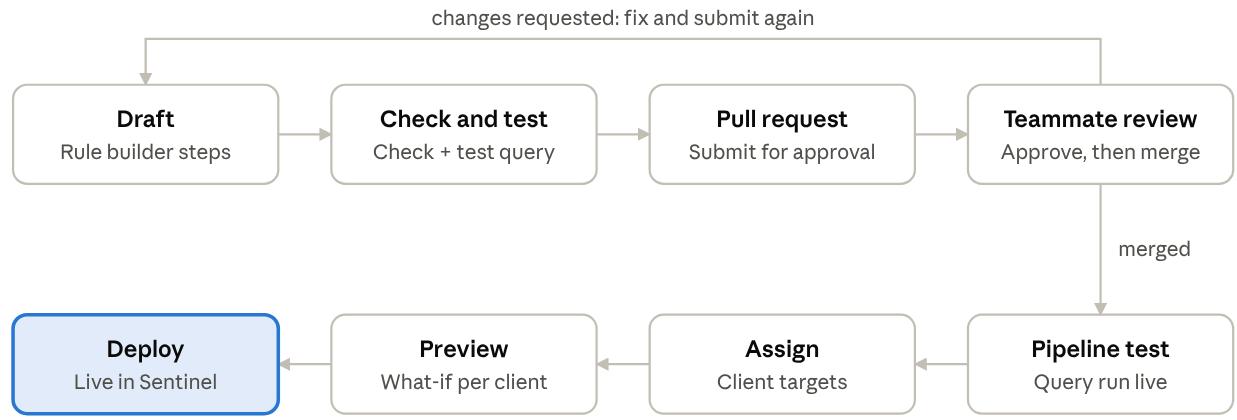
Nov 12, 2026 · @Adam Ring · Sentinel Workspace 2.3.0, Rule builder

1. Purpose, scope and the rule lifecycle

This SOP is the standard way to author, test and submit a Microsoft Sentinel analytics rule with the **Rule builder** in the Sentinel Workspace app. It covers new rules and changes to rules already in the detection library (isi-security/detection-as-code). Signing in, browsing the library and deploying to clients are in **SOP: Sentinel Workspace desktop app**.

The builder writes one YAML rule file in the repository's schema. It supports **Scheduled** and **NRT** (near real-time) rules. Nothing you do in the builder changes Sentinel: a rule reaches a client only after a teammate approves its pull request, it merges, the pipeline tests it, and an engineer assigns, previews and deploys it.

A rule reaches Sentinel only after review, merge and a passing pipeline



Top row: this SOP. Bottom row: engineers, from the Rule library (app SOP, Procedure 5).

rule lifecycle · 8 stages, 1 loop back

A reviewer who asks for changes sends the rule back to the draft; only a merged rule that passes the pipeline can be assigned, previewed and deployed.

2. Before you start

- [ ] Sentinel Workspace 2.3.0 or later installed, and **Check this PC** all OK
- [ ] Signed in to GitHub with write access to isi-security/detection-as-code (the Rule builder toolbar shows **GitHub: <your account>**)
- [ ] Rule library set on Connect (Organization **isi-security**, Repository **detection-as-code**)

- [ ] For **Run test query**: signed in to the client on Connect with a workspace chosen, and Log Analytics Reader or Microsoft Sentinel Reader on its resource group
- [ ] A ticket or request that describes what the rule must detect, and the data table it reads

If the toolbar pill says **GitHub: not signed in**, select **Sign in to GitHub**; if it says **Rule library not set**, select **Go to Connect**. You can write and save a draft without either, but you cannot submit.

3. The Rule builder at a glance

Open **Rule builder** in the sidebar (Ctrl+3). The toolbar holds the ways in and out of a draft:

Button	What it does
New rule (Ctrl+N)	Starts an empty rule with a new rule ID
Edit existing rule…	Opens the library, loaded from GitHub, to pick the rule to change
Open…	Recent drafts, **Open draft file…**, or **Open rule file…** for a .yml file on disk
Save draft (Ctrl+S)	Keeps the draft under its rule name in `desktop-data\drafts`
Check rule	Checks every field against the repository schema and shows the rule file
Export rule file…	Checks the rule, then saves the .yml file on this PC
Submit for approval	Compares with the library and opens a pull request for review
My submissions…	Rules you submitted and their approval status

The form has five steps, listed down the left: **Basics**, **Query & schedule**, **MITRE ATT&CK**, **Advanced** and **Review**. Move with **← Back** and **Next →** at the foot of the form (Ctrl+Page Up / Ctrl+Page Down) or select a step in the list.

The pill beside the step title shows the draft state:

State	Meaning
New draft	Started, nothing saved yet
Unsaved changes	Edited since the last save or check
Saved draft	Saved on this PC; not checked since
Checked	Passed Check rule; Review shows the generated file
Submitted	A pull request exists for this draft

4. Step 1 — Basics

Basics names the rule and sets who owns it and how serious its alerts are.

1. Select **New rule** (Ctrl+N).
2. Fill in each field:

Field	Rule	Team standard
Rule name	Required	Short and specific, for example *Brute force against Entra ID accounts*; prefix test rules with `Test -`
Description	Required, at least 20 characters	What it detects, why it matters and what the analyst should check first
Severity	Informational, Low, Medium or High	Match the impact if the alert is true
Owner	Lowercase team or account ID: letters, digits and `. _ @ -`	Your team ID, for example `soc-team`
Lifecycle	Baseline, Production or Retired	**Baseline** for every new rule; Production only once tuned; Retired rules must stay disabled

Field	Rule	Team standard
Rule type	Scheduled or Near real-time	Scheduled unless the detection must fire within about a minute
Enable the rule when it is deployed	Off by default	Leave off unless the change is approved to go live enabled
Create incidents from alerts	Optional	Tick when each alert should raise a Sentinel incident
Rule ID	Generated, read-only	Never change it; edits keep the library rule's ID

5. Step 2 — Query & schedule

This step holds the KQL and when it runs. Get the query right here: it is the part reviewers read most closely and the part the pipeline tests live.

Write the query

1. Type or paste the KQL into the **Query (KQL)** box. Start from the table, filter early with `where`, and project only the columns the alert needs.
2. Use the editor's help as you type:
   - **Colours** mark steps, tables, functions, operators, quoted text, numbers and comments. **Theme**, under the box at the right, switches between Daylight, Midnight and Neon.
   - **Completions** offer what fits at the caret: steps after a `|`, tables where a query starts or joins, and the common columns of the tables the query reads. Initials work (`upn` finds `UserPrincipalName`). **Up/Down** choose, **Tab** inserts, **Esc** closes, **Ctrl+Space** opens the list anywhere.
   - **Tab** indents and **Ctrl+Tab** moves to the next field, as in Log Analytics.
3. Read the line under the box. The **KQL check** runs as you type and underlines each problem with its line and column:
   - **Red** — KQL always rejects it: a quote or bracket left open, a `|` with nothing before or after it. Check rule and Run test query stop on these.

- **Amber** — a likely slip that doesn't block: an unknown step (`| wher`), a step in capitals, `=` instead of `==` , `&&` or `||` instead of `and` / `or` , or a table, function or operator a letter or two off (`SignLogs` , `tostrng(` , `contians`).

4. Fix slips in one click: the link beside the line (**Change to where**, or **Fix all 3**), right-click an underlined word > **Change to…**, or **Ctrl+.** with the caret on it. **Ctrl+Z** undoes it.

Custom tables, your own functions and watchlists are never flagged. Completions list the usual columns of well-known tables, not the workspace's own schema, so type a custom column once and it is offered after that.

Set the schedule (Scheduled rules)

Field	Allowed	Guidance
Run every	5 minutes to 14 days, a number and a unit	How often the query runs
Look back	At least Run every, up to 14 days	How much data each run reads; usually the same as Run every, longer for slow-arriving logs
Alert when the number of results	Is greater than, less than, equal to or not equal to	Usually *greater than*
Threshold	A whole number, 0 or more	Usually 0

Near real-time rules have no Run every, Look back or threshold; the rule file leaves them out.

Test the query

1. Select **Run test query** (or **Shift+Enter** in the Query box). It runs the query read-only against the workspace chosen on Connect, over the rule's Look back, at most the last day (NRT rules use the last day). The line beside the button names the period.
2. Read the result: how many rows came back, how long it took and the first 50 rows. Point at a shortened value to see it in full; **Ctrl+C** copies selected rows.
3. Tune until the result matches what should alert. A rule that returns hundreds of rows a day will flood the queue; tighten its filters or raise its threshold.

4. If Log Analytics rejects the query, lacks permission, or the query runs longer than 2 minutes, the reason appears under the button with **Copy details**. An unknown table that is a letter or two off a common one is named for you.

Editing the query or Look back marks the results **Out of date**; run it again. The button is greyed, with the reason beside it, until you are signed in with a workspace chosen, and for GCC High workspaces (test those in the portal). Results are never saved, and nothing in the workspace changes. Test against a client whose data the rule is meant for.

6. Step 3 — MITRE ATT&CK

Every rule needs at least one tactic and one parent technique; reviewers and coverage reports rely on them.

1. Tick every **Tactic** the rule detects (for example *CredentialAccess*).
2. Enter **Parent techniques** as IDs such as `T1110` (Enterprise) or `T0849` (ICS).
3. Enter any **Sub-techniques** as `T1110.003`, and list its parent `T1110` under Parent techniques as well.
4. Tick **Allow missing MITRE details for a migrated rule** only for an approved migration of an existing rule with no mapping. Each destination client target must also permit it.

7. Step 4 — Advanced

Additional rule properties holds optional settings as YAML. Leave `{}` for a simple rule. Use it for entities, details and grouping:

Property	Type	Use it to
`entityMappings`	list	Map query columns to entities (Account, Host, IP…) so incidents link to them
`customDetails`	mapping	Surface key columns on the alert
`alertDetailsOverride`	mapping	Build the alert name or description from columns
`eventGroupingSettings`	mapping	One alert per result, or one per run

Property	Type	Use it to
`groupingConfiguration`	mapping	Group alerts into incidents
`suppressionEnabled`, `suppressionDuration`	true/false, ISO duration (`PT1H`)	Stop repeat alerts for a period after one fires

When you edit an existing rule, properties already in its file are kept as they are. Keys outside the repository schema are refused by Check rule.

8. Step 5 — Review, check and save

1. Select **Save draft** (Ctrl+S) at any point. The draft is kept under its rule name in `desktop-data\drafts` and listed under **Open…**.
2. Select **Check rule**. It checks every field against the repository schema at once:
   - On success the banner turns green, the pill shows **Checked**, and **Review** shows the generated rule file. Read the whole file once.
   - On failure the builder outlines every field that needs fixing in red, explains the fix under each one and opens the first step with a problem. A red outline clears as soon as you edit the field.
3. Continue with Submit for approval (Section 10), or select **Export rule file…** to save the .yml locally.

Any edit after a check clears the checked file; Submit for approval checks again automatically. Closing the app with unsaved changes asks first. Every 30 seconds the builder also keeps a recovery copy of unsaved work; if the app or PC stops, the next start asks **Restore unsaved rule?** — **Restore** continues, **Discard** deletes it, **Esc** keeps it for next time.

9. Editing an existing rule

Start every edit from the current library version, so the review shows exactly what you changed. No git knowledge is needed.

1. Open the rule:
   - **From the library (recommended):** select **Edit existing rule…**. The library loads from GitHub; double-click the rule, or select it and choose **Edit selected rule**.

    - **From a rule file: Open… > Open rule file…**. When its rule ID is in the library, the banner reads *Matched "…" in the library* and it is submitted as an update to that rule; otherwise it is treated as a new rule.
2. Make the change in the relevant step. Do not change the Rule ID — the review rejects an edit that alters it.
3. Run the test query again if the query, schedule or Look back changed.
4. Select **Check rule**, then **Submit for approval** (Section 10). The file path is filled in for you.

An opened file may be older than the library version. The review then warns you to check the red (removed) lines: a red line you did not mean to remove is someone else's newer change — close the window and start again from **Edit existing rule…**. Do the same if the review says the library changed since you opened the rule.

To retire a rule, set Lifecycle to **Retired** and leave **Enable the rule when it is deployed** off, then submit as an update.

10. Submit for approval and track it

Submitting sends the rule to a teammate; nothing deploys until it is approved, merged and passed by the pipeline. The app does the git steps.

1. Select **Submit for approval**. The rule is checked, then the Submit for approval window compares it with the library (about 15 seconds). Nothing is sent to GitHub yet.
2. Read the window from the top:
    - **Update to an existing rule** or **New rule** — confirms what you are doing.
    - The progress strip: 1 Check → 2 Submit → 3 Teammate approves → 4 Deployed.
    - The banner: green is ready, amber lists things to check, red says what to fix.
    - Library, **Submitting as** (your GitHub account), Rule file and **Used by clients**.
    - **What will change**: added lines in green, removed in red, plus the clients that use the rule. For a new rule this also shows the deploy workflows (`sentinel.yml`, `sentinel-multi-workspace.yml`) gaining it in their rule dropdowns.
3. If the banner is red, follow its advice. Common ones: the rule already exists (use **Edit existing rule…**), no write access (ask a repository admin), nothing changed, or a file name already used in another folder (rename the rule file under **Where it will be saved**).

4. Open **Where it will be saved** only if asked to change the repository or file name, then **Check again**.
5. Fill in **Why is this changing?** — one or two sentences for the reviewer, with the ticket number. It is required and becomes the pull request's description.
6. Select **Submit for approval**. The banner turns green with the link. Select **Open in GitHub**, check the pull request and its CI, and ask a teammate to approve it.
7. After approval and merge, the pipeline tests the query live. An engineer then assigns, previews and deploys it (SOP: Sentinel Workspace desktop app, Procedure 5).

Follow every submission in **My submissions…**, which asks GitHub for its state:

Status	What to do
○ Not finished	**View changes** and finish the submission
… Waiting for a teammate's review	Nothing; ping your reviewer if urgent
⚠ Changes requested	Read the comments in GitHub, fix the rule, submit again
✓ Approved — ready to merge	An engineer merges it
✓ Merged	Done; the pipeline takes it from here
✕ Closed without merging	Read the comments before resubmitting

If GitHub times out during submit, a write may have completed: open the unfinished one in My submissions and submit again; do not start a new submission.

11. Validation rules reference

Check rule applies the repository's own schema. A rule passes only when all of these hold:

Field	Must be
Rule ID	A non-zero GUID, unchanged on edits
Name, Owner, Query	Not empty
Description	At least 20 characters

Field	Must be
Owner	Lowercase letters, digits and `. _ @ -`, starting with a letter or digit
Severity	Informational, Low, Medium or High
Lifecycle	baseline, production or retired; retired rules disabled
Enabled, Create incidents, Suppression	true or false
Rule type	Scheduled or NRT
Run every (Scheduled)	5 minutes to 14 days
Look back (Scheduled)	At least Run every, at most 14 days
Threshold (Scheduled)	A whole number, 0 or more
NRT rules	No Run every, Look back, operator or threshold
Tactics	From the ATT&CK list in Step 3; at least one, unless the migration exception applies
Techniques	`T` + 4 digits; at least one, unless the migration exception applies
Sub-techniques	`T` + 4 digits + `.` + 3 digits, with its parent listed
Query	Does not end in a dangling `
Advanced properties	Only schema fields, each of the right type

Passing Check rule is not proof the rule works: Run test query and the pipeline's live test are.

12. Checklist, troubleshooting and shortcuts

Before you submit:

- [ ] Name and description say what the rule detects and why; test rules start `Test -`
- [ ] Lifecycle **Baseline** for a new rule, and **Enable the rule when it is deployed** off unless approved
- [ ] No red or amber KQL check marks left unexplained

- [ ] **Run test query** run against a relevant client, with a sensible number of rows
- [ ] Tactics and techniques mapped
- [ ] **Check rule** passed and the Review file read
- [ ] **Why is this changing?** names the ticket

Symptom	Likely cause	Action
Check rule fails	A field is missing or invalid	Fix each field outlined in red; the fix is under it
Red mark under the Query box	A quote or bracket left open, or a stray `	`
Run test query is greyed	Not signed in, no workspace, or GCC High	Sign in and choose the workspace on Connect; test GCC High in the portal
Run test query: permission denied	No Log Analytics Reader or Sentinel Reader	Ask an engineer for read access, or test in the portal
Test query: table not found	Table missing in this workspace or misspelt	Check the suggested name; confirm the client collects that table
"There are no file changes to submit"	Draft matches the library file	Make the intended change, check again
Review blocked: file exists or source is stale	Editing outside the library, or the library changed	Start again from **Edit existing rule…**
Review refused: file name used in another folder	The deploy dropdowns could not tell the two apart	Rename the rule file under **Where it will be saved**
GitHub: not signed in	GitHub CLI has no account	**Sign in to GitHub** in the toolbar
"Restore unsaved rule?" at start	App or PC stopped before saving	**Restore** to continue, or **Discard**

Shortcuts: **Ctrl+N** new rule · **Ctrl+S** save draft · **Ctrl+Page Down / Up** next or previous step · **Shift+Enter** run test query · **Ctrl+Space** completions · **Ctrl+.** apply the suggested spelling · **Ctrl+Z** undo · **Tab** indent, **Ctrl+Tab** next field (in the Query box).